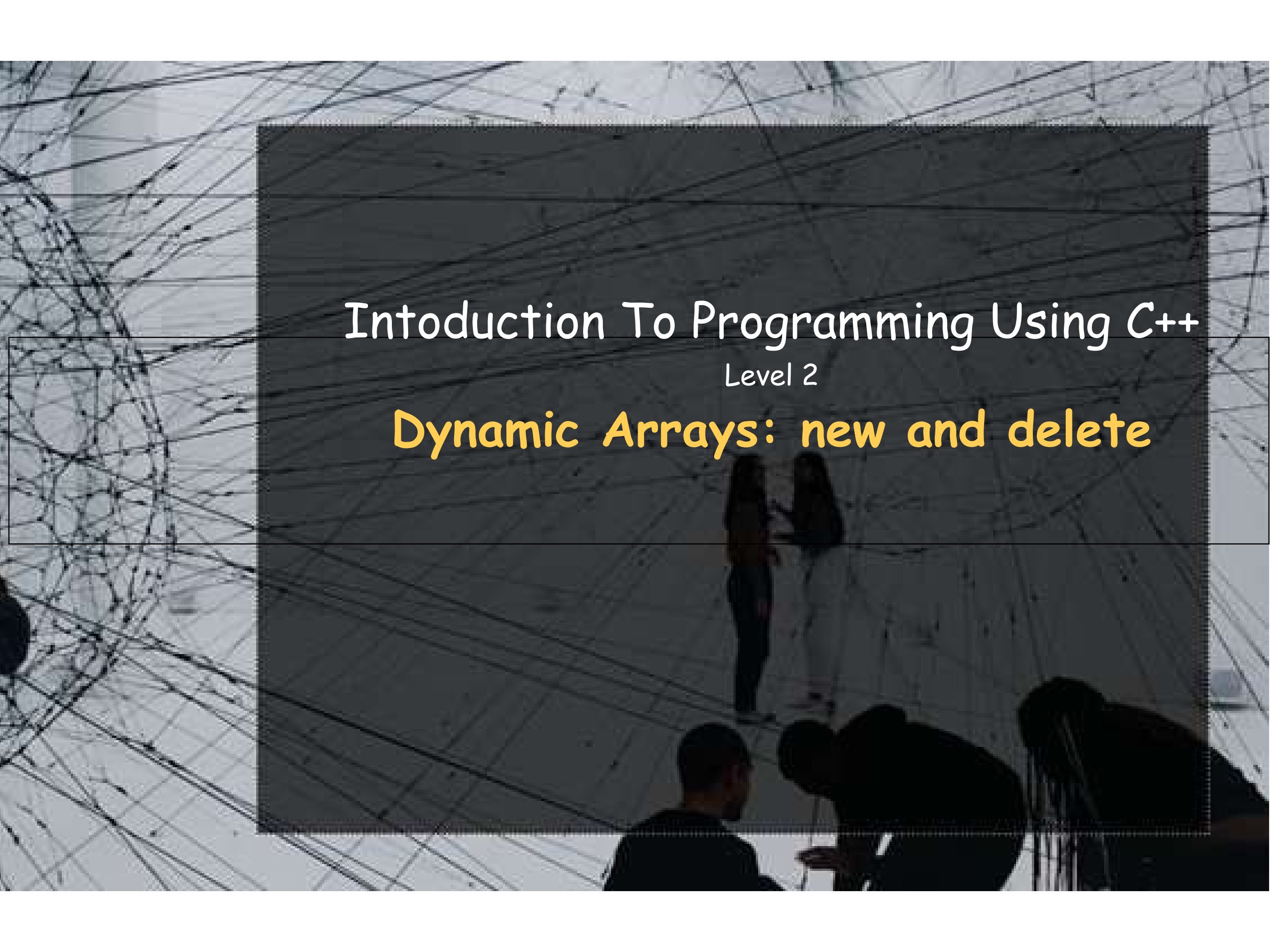


Introduction To Programming Using C++

Level 2

Dynamic Arrays: new and delete



Introduction To Programming Using C++

Level 2

Dynamic Arrays: new and delete

Definition

Dynamic Arrays

 Dynamic Arrays are arrays whose size is determined during run-time using a pointer and the following operations:

 ``new[]`` → to allocate memory for a number of elements.

 ``delete[]`` → to deallocate the array after finishing with it.

Explanation

-  The main advantage is that you do not need to determine the array size in advance.
-  You can allocate the number of elements based on user input or program conditions.
-  This allows the program to allocate only the memory it needs at run-time.

💡 Example

```
int main()
{
    int num;
    cout << "Enter total number of students: ";
    cin >> num;
    float* ptr;
    // Allocate dynamic memory for the number of students
    ptr = new float[num];
    cout << "Enter grades of students:" << endl;
    for (int i = 0; i < num; i++)
    {
        cout << "Student " << i + 1 << ": ";
        cin >> *(ptr + i);
    }
    cout << "\nDisplaying grades of students:" << endl;
    for (int i = 0; i < num; i++)
    {
        cout << "Student " << i + 1 << ": "
              << *(ptr + i) << endl;
    }
    // Deallocate the dynamic array
    delete[] ptr;
```

Note

- ◆ Modern languages such as Java and Python do not normally expose direct pointer manipulation like C++.
- ◆ C++ provides this low-level control, but it requires careful memory management.
- ◆ In modern C++, containers such as `std::vector` are often preferred over manually managed dynamic arrays.

⊘ Warning

- ◆ Forgetting to use ``delete[]`` can cause a memory leak.
- ◆ Do not use regular ``delete`` with a dynamically allocated array.
- ◆ For memory allocated with ``new[]``, always use ``delete[]``.
- ◆ Incorrect pointer usage can lead to serious memory errors and undefined behavior.

✦ Summary

- ✓ ``new[]`` is used to create a dynamic array with a size determined at run-time.
- ✓ ``delete[]`` is required to release the memory allocated by ``new[]``.
- ✓ The array size can be determined based on user input or program conditions.
- ✓ Elements can be accessed using ``ptr[i]`` or ``*(ptr + i)``.
- ✓ Dynamic arrays provide flexibility, but manual memory management requires care.

Thank you 😊